

Personal Academic Website

Maintenance, Updating & GitHub Hosting Guide

Website of Md Zulquar Nain — built with Quarto, R, and one Excel file

August 2026

Contents

1	How the website is organised	2
2	Updating content: one Excel file	2
2.1	Typical tasks	3
2.2	Writing a blog post	3
3	Course pages	3
3.1	How the Teaching page is put together	3
3.2	Adding a new course	4
3.3	Linking to slides and data	4
3.4	Publishing a folder of pre-built slides	4
3.5	Rebuilding a slide deck	5
3.6	Size warning	5
4	Rendering	6
4.1	From RStudio (easiest)	6
4.2	From a plain terminal	6
4.3	Updating the PDF CV	6
5	Google Scholar, OpenAlex, and caching	6
6	The Data Corner (MoSPI)	6
6.1	Knobs	7
6.2	Why the series are chain-linked	7
6.3	Other Indian sources	7
7	The inflation tracker	7
7.1	Why it is not automated	7
7.2	The monthly routine	8
7.3	Three parts that must stay in step	8
7.4	The notifier	8
7.5	Why the page is not self-contained	8
8	Customising the look	9
8.1	The animated background (<code>background.js</code>)	9
8.2	The estimated IRF surface (<code>irf-surface.js</code>)	9
8.3	Site styling (<code>styles.css</code>)	10
8.4	Checking how a change looks	11
8.5	Section formatting (<code>R/helpers.R</code>)	12

8.6 Courses taught	12
9 Hosting on GitHub Pages	12
9.1 Choosing the repository name — this fixes the URL	12
9.2 First-time setup	12
9.3 Every update afterwards	14
9.4 What to commit	14
9.5 Visitor analytics	14
10 Troubleshooting	15

1 How the website is organised

The project lives at `C:\Users\Abc\Desktop\academic-website`. **Do not move it back into a Google Drive folder:** Drive’s sync client locks the `_site` output folder during rendering and the render fails with *PermissionDenied* errors.

Path	What it is
<code>_quarto.yml</code>	Site configuration: navbar, footer, fonts, excluded folders
<code>index.qmd</code>	Homepage (hero, interests, selected papers, data corner)
<code>research.qmd</code>	Publications, projects, presentations (all from Excel)
<code>teaching.qmd</code>	Teaching statement, course cards, courses taught, advising
<code>courses/</code>	One <code>.qmd</code> per course website (hand-written, not Excel)
<code>Rclass_2025/</code>	Pre-built ECM2073 slide deck, served as-is at <code>/Rclass_2025/</code>
<code>cv.qmd</code>	Web CV (awards, workshops, skills from Excel)
<code>blog.qmd + posts/</code>	Blog listing and the posts themselves
<code>R/helpers.R</code>	All shared code; every page only calls functions from here
<code>R/pre-render.R</code>	Runs before each render; refreshes <code>cv.pdf</code> from the CV project
<code>cvnain/</code>	The separate Quarto project that builds the PDF CV
<code>cvnain/data/cv_data.xlsx</code>	The single source of truth (see below)
<code>background.js</code>	The animated homepage background
<code>irf-surface.js</code>	Generated: the estimated IRF surface it draws (Section 8)
<code>styles.css</code>	Palette, fonts, cards, badges, reading measure
<code>images/</code>	Profile photo and other pictures
<code>archive/</code>	Not needed to build or host the site; kept, never committed
<code>.github/workflows/publish.yml</code>	Auto-deployment to GitHub Pages
<code>.github/workflows/check-inflation-data.yml</code>	Monthly nag when the inflation tracker falls behind
<code>software/</code>	Shiny splicer app and the inflation tracker
<code>_site/</code>	Rendered output — never edit by hand

2 Updating content: one Excel file

`cvnain/data/cv_data.xlsx` drives *both* the PDF CV and the website. Each sheet feeds specific parts of both documents:

Sheet	Feeds on the website	Feeds in the PDF CV
publications	Research page + homepage cards	Publications section
presentations	Research page	Presentations / Conferences
projects	Research page	Projects
teaching_courses	Teaching page (by level)	Teaching section (by level)
advising	Teaching page	Advising
awards	CV page	Scholarship / Awards
workshops	CV page	Workshops and Training
service	CV page	(currently CV-off)
memberships	CV page	Academic Affiliation
skills	CV page	Skills
scholar_fallback	Stats badges (fallback tier)	Stats box (fallback tier)
software	Software page cards (incl. <code>status_file</code>)	—

2.1 Typical tasks

Add a new paper. Add a row to the *publications* sheet (`category = journal, submitted, or working`; fill `authors, year, title, journal, doi`). Re-render (Section 4).

Choose the papers featured on the homepage. Add a column named `selected` to the publications sheet and put YES on up to three rows. Without it, the three most recent journal articles are shown.

Update Scholar fallback figures. Edit the *scholar_fallback* sheet occasionally (`citations, h_index, i10_index, updated`). It is used whenever Google Scholar blocks automated access (which is common — see Section 5).

Things not in Excel (they rarely change): hero text and interest cards in `index.qmd`; education and appointments in `cv.qmd`; the teaching statement in `teaching.qmd`; everything on the course pages in `courses/` (Section 3).

Why course pages are not in Excel. Excel earns its place when the same rows feed *two* outputs — `publications` and `teaching_courses` print into both the website and the PDF CV, so one edit updates both. Course material feeds one page only, and a course page is mostly prose (syllabus, prerequisites, assessment) that no spreadsheet row can hold. So courses are plain `.qmd` files.

2.2 Writing a blog post

Create a folder under `posts/` with an `index.qmd`:

```
posts/my-new-post/index.qmd
---
title: "My new post"
description: "One-line teaser shown in the listing."
date: 2026-08-01
categories: [economics, teaching]
---
Body of the post in ordinary markdown...
```

It appears on the Blog page automatically at the next render.

3 Course pages

3.1 How the Teaching page is put together

`teaching.qmd` has four parts, top to bottom:

1. The **teaching statement** — ordinary prose at the top of the file. Edit it directly.
2. **Course Websites** — a card for every `.qmd` in `courses/`. This list is *generated*; there is nothing to maintain.
3. **Courses Taught** tables — from the *teaching_courses* Excel sheet, unchanged.
4. **Advising** — from the *advising* Excel sheet, unchanged.

The cards come from the `listing:` block in the front matter of `teaching.qmd`, the same mechanism the Blog page uses for `posts/`. It scans `courses/` and builds one card per file, newest date first.

3.2 Adding a new course

Create one file, `courses/my-course.qmd`. Everything in the front matter feeds the card on the Teaching page:

```
---
title: "Financial Econometrics"
subtitle: "ECM3081 . M.A. Economics . Aligarh Muslim University"
description: "One or two lines describing the course. This is the
  paragraph shown on the Teaching page card."
date: 2026-01-10          # newest date sorts to the top
image: /images/course-fin-econ.jpg
categories: [R, Econometrics, PG]
toc: true
---

## About the course
Ordinary markdown from here down: syllabus, readings, links...
```

Render, and the Teaching page card appears on its own. **There is no list to update anywhere** — which is the point: the index can never fall out of step with the pages that actually exist.

Put the card image in `images/` and refer to it as `/images/name.jpg` (the leading slash means “from the project root”). Roughly 800×500 pixels works well. If you omit `image:` entirely the card still renders, just without a thumbnail.

3.3 Linking to slides and data

Course material lives in its own folder at the top level of the project (`Rclass_2025/` is the first one). Because the course page sits one level down in `courses/`, links start with `../`:

```
[Open](<../Rclass_2025/25_Lecture 1 Introduction to R.html>)
```

The angle brackets matter. Your slide filenames contain spaces. Ordinary markdown link syntax breaks on a space; wrapping the path in `<...>` tells Pandoc to take the whole thing as one URL, so you can paste the filename exactly as it appears in Windows Explorer and never think about `%20`.

Capitalisation matters too. Windows treats `25_lecture 1...` and `25_Lecture 1...` as the same file; the Linux server behind GitHub Pages does not. A link that works locally can 404 once published. Copy filenames, do not retype them.

3.4 Publishing a folder of pre-built slides

`Rclass_2025/` was rendered in its own RStudio project and its HTML is finished. Two entries in `_quarto.yml` publish it untouched:

```
render:
- "!Rclass_2025/"          # do NOT re-render the .qmd/.Rmd inside
resources:
- Rclass_2025/**          # DO copy the finished HTML and assets
- "!Rclass_2025/_site/**" # skip the duplicate copy of the same deck
```

The `render` exclusion is essential: without it Quarto tries to rebuild all sixteen slide decks on every site render, which is slow and fails on the two `xaringan` decks. To add a second course folder later, copy both lines and change the name.

3.5 Rebuilding a slide deck

Most of the ECM2073 decks are Quarto `revealjs`. Two of them (Lecture 8 *R Base Plot* and Lecture 12 *ggplot2*) are older `xaringan` decks written in R Markdown, and they are built by `rmarkdown`, not Quarto. From the `Rclass_2025` folder:

```
$env:RSTUDIO_PANDOC = "C:\Program Files\RStudio\resources\app\bin\quarto\bin\tools"
& "C:\Program Files\R\R-4.5.2\bin\Rscript.exe" -e "rmarkdown::render('25_Lecture 8
R Base Plot.Rmd')"
```

The `RSTUDIO_PANDOC` line is the part people miss: R on its own cannot find Pandoc outside RStudio and fails with “*pandoc version 1.12.3 or higher is required*”. Inside RStudio just press **Knit** and none of this applies.

These decks need `xaringan`, `tidyverse`, `skimr`, `janitor`, `DT`, `here`, `kableExtra` and `titanic`. They write a shared `Rclass_2025/libs/` folder plus a `._files/` folder of figures — both must be committed alongside the HTML or the published slides lose their styling and images.

3.6 Size warning

`Rclass_2025/` used to be dominated by two screencasts. Both were recorded at a fixed high bitrate (17.6 and 8.3 Mbit/s) on static screen content that needs a small fraction of it, and both have been re-encoded in place at `crf 28` with no change of resolution, frame rate or duration:

File	Was	Now
images/dataimport.mp4 (Lecture 5)	86 MB	0.9 MB
images/project1.mp4 (Lecture 2)	50 MB	1.1 MB
Everything else		≈108 MB
Folder total	244 MB	≈110 MB

Quality was checked before the swap: SSIM 0.998 and PSNR 46 dB against the originals, and RStudio console text is indistinguishable at 1:1. The originals were moved (not deleted) to `academic-website-video-originals/` next to the project folder.

The whole published site now comes to about **233 MB** (`Rclass_2025/` 110 MB, the Shinylive splicer app 96 MB, the inflation tracker’s libraries 21 MB, everything else 6 MB), against the **1 GB** that GitHub Pages asks a published site to stay under.

What the *repository* carries is smaller again. Git stores blobs by content, so the sixteen `._files/` folders in `Rclass_2025/` — 1408 files, but only 290 distinct ones, because every deck ships its own copy of `reveal.js`, `Font Awesome` and `Source Sans Pro` — cost about 30 MB once, not 110 MB. There is no reason to consolidate them by hand.

Across the whole project the tracked working tree is about 145 MB but only 53 MB of it is distinct content, and that — not the 145 — is what the first push actually carries.

No file in the project now exceeds 50 MB, the threshold at which GitHub starts warning; the hard rejection limit is 100 MB. The 247 MB duplicate render of the deck has been moved to `archive/Rclass_2025/_site/` and can be deleted from disk entirely whenever you like.

4 Rendering

4.1 From RStudio (easiest)

Open `academic-website` as the working directory and run `quarto render` in the Terminal pane (or use Build → Render Website). RStudio finds R automatically.

4.2 From a plain terminal

The RStudio-bundled Quarto needs to be told where R lives:

```
$env:QUARTO_R = "C:\Program Files\R\R-4.5.2\bin"
& "C:\Program Files\RStudio\resources\app\bin\quarto\bin\quarto.exe" render
```

Use `quarto preview` instead of `render` for a live-reload local server while editing.

4.3 Updating the PDF CV

Render the CV inside its own project first, then the site:

```
cd cvnain
quarto render "Dr Nain_CV.qmd"      # produces Dr-Nain_CV.pdf
cd ..
quarto render                       # pre-render copies it to cv.pdf
```

5 Google Scholar, OpenAlex, and caching

The stats badges (Citations, h-index, i10-index) are resolved in this order at render time:

1. **Live scrape** of the Scholar profile (ID 7Db2EHMAAAAJ). Google frequently answers scripted requests with a CAPTCHA, in which case this tier fails silently.
2. **Cache** of the last successful live fetch (`data/scholar_stats_cache.csv`) — written automatically whenever tier 1 succeeds. *Commit these files to git* so the GitHub build (whose datacenter IPs are always blocked) can use them.
3. **Excel** `scholar_fallback` sheet.
4. **OpenAlex** (author ID A5034570980), a free API that is never blocked — used only if the Excel values are missing. Note its counts are lower than Scholar's (different coverage).

6 The Data Corner (MoSPI)

The homepage chart shows **monthly headline CPI inflation** and **IIP growth** for All-India, fetched at render time from the Ministry of Statistics and Programme Implementation's open API (`api.mospi.gov.in`) — the same backend as the [eSankhyiki portal](#), and the same one `software/inflation-tracker.qmd` uses. **No API key is needed.** Each successful fetch caches to `data/mospi_cache.csv`, which is used if the API is unreachable, so commit that file.

It replaced an annual World Bank series in August 2026: World Bank data for India lags by a year or more, while MoSPI publishes CPI around the 12th of each month. `data/wdi_cache.csv` is left over from that and can be deleted.

6.1 Knobs

In R/helpers.R	Effect
ECON_YEARS	How many years of history the chart shows (currently 5)
cpi_series()	Which CPI cut is plotted — currently All-India, Combined sector, headline
iip_series()	Currently the General index; type can also be <i>Sectoral</i> or <i>Use-based category</i>
smoothing in econ_chart()	Curve of the line: 0 straight, 1 current, 1.3 maximum

6.2 Why the series are chain-linked

MoSPI republishes each index on a new base every few years, and the old and new series *disagree where they overlap*. The base 2022–23 IIP puts March 2024 growth at **19.0%**; base 2011–12 puts the same month near **5%**, because the new base’s first year is measured against a partial history. Taking “the newest base wins” would draw that 19% spike as though it were real.

`chain_bases()` therefore takes the *oldest* base first and lets each newer base contribute only the months the previous one does not reach: the long-running series carries the history, the newest one extends the tip, and the join sits at the end where the discontinuity is smallest. If you add another indicator, route it through the same function.

Not every spike is an artifact. The chart’s May 2022 IIP reading of 19.7% is genuine — a base effect against the 2021 Delta-wave lockdown — and is left in.

6.3 Other Indian sources

If you want to add series later, in rough order of how pleasant they are to work with:

Source	Covers	Key?
MoSPI API	CPI, IIP (1994–), ASI; monthly	no
data.gov.in	Very broad catalogue; quality varies a lot by dataset	free key
RBI DBIE	Monetary, banking, BoP, fiscal — the richest, but no documented public API; scraping breaks on redesign	—
World Bank	Cross-country, annual, lagged	no
UN Comtrade	Bilateral trade (<code>comtrade</code>)	free key
BIS	Policy rates, REER, credit-to-GDP	no

7 The inflation tracker

`software/inflation-tracker.qmd` is a standing dashboard of Indian retail inflation, published on the Software page as the second card. It is **rendered by hand**, not by the site build.

7.1 Why it is not automated

It needs `sf` (and therefore GDAL, GEOS and PROJ as system libraries), plus `ggiraph`, `ragg` and `DT`. Installing that on a GitHub runner turns a four-minute build into fifteen, for a page that changes *twelve times a year*. More to the point, MoSPI’s API is young — base years and field names shift — and an automated publish fails *silently*: the build goes green and a broken

page sits on your site. Rendering by hand keeps you looking at the state map at the moment things break.

7.2 The monthly routine

MoSPI publishes CPI around the 12th at 16:00 IST. After that:

```
cd software
quarto render inflation-tracker.qmd
cd ..
quarto render
git add . ; git commit -m "Inflation tracker: <month> data" ; git push
```

Look at the state map before pushing. It is the first thing to break when a field name changes.

7.3 Three parts that must stay in step

Piece	Role
software/tracker_status.json	Written on every tracker render: latest data month, headline rate, render date
print_software()	Reads it via the <code>status_file</code> column in the <i>software</i> sheet and stamps “Data through Jul 2026” on the card, so a stale page announces itself
.github/workflows/ check-inflation-data.yml	Compares it against MoSPI daily on the 12th–18th and opens an issue when the page is behind

The status file is the hinge: rename a field in it and both the card stamp and the notifier stop working. It must be committed.

7.4 The notifier

The Action *checks but never publishes*. It runs at 12:00 IST daily from the 12th to the 18th, asks MoSPI for the newest CPI month carrying an inflation figure, and compares it with `tracker_status.json`. If the site is behind it opens a labelled issue with the rebuild commands; it will not open a second issue for the same release. Nothing else in the repository depends on it, so if it ever becomes a nuisance, delete the file.

Run it on demand from the repository’s *Actions* tab (*Run workflow*) to test it.

7.5 Why the page is not self-contained

`embed-resources` is deliberately `false`. Self-contained, the tracker is one 11 MB file, and a *fresh* 11 MB blob enters git history at every monthly rebuild — about 130 MB a year, permanently. Split out, the DataTables and `ggiraph` JavaScript in `inflation-tracker_files/libs/` (20 MB) is byte-identical each render and git stores it once; only the six charts churn, about 1 MB a month. The cost is a one-off 20 MB and the rule that `inflation-tracker_files/`, `tracker.css` and `tracker_status.json` must all be published alongside the HTML — they are listed in `_quarto.yml` under `resources`.

Two MoSPI endpoints exist but were not usable as of August 2026: `/api/nas/getNASData` (national accounts) returns HTTP 500 for every parameter combination tried, and `/api/asi/getASIData` wants a `classification_year` whose accepted values are undocumented. Worth re-probing

later — the API tells you what it wants if you call it with no parameters and read the error body.

8 Customising the look

8.1 The animated background (background.js)

Four layers share one full-page canvas: a drifting network of variable symbols, floating equations, small 2-D impulse-response panels that draw themselves and respawn, and the 3-D surface. Everything is controlled by the `STYLE` block at the top of the file:

Knob	Effect
<code>intensity</code>	Master visibility: 0.5 subtle, 1 current, 2 bold
<code>speed</code>	Master animation pace
<code>palette</code>	Five [R,G,B] colours used by equations, nodes, panels
<code>surface.size / posX / posY</code>	Size and position of the 3-D surface
<code>surface.faceAlpha</code>	Opacity of the shaded faces. Raise for a more solid surface, lower if it competes with the hero text
<code>surface.edgeAlpha</code>	Opacity of the mesh drawn over the faces
<code>surface.colormap</code>	"parula" (MATLAB's default) or "site" to use the page palette instead
<code>surface.azimuth / elevation</code>	The fixed viewpoint. Currently MATLAB's <code>view(-46, 29)</code> . There is no spin
<code>surface.evolve</code>	How fast the <i>fallback</i> surface reshapes; unused when real data is present
<code>EQUATIONS / LABELS</code> arrays	The floating formulas and node symbols

The surface is drawn like MATLAB's `surf()`: faces filled from the colormap with a mesh over them, painted back to front. Colour encodes the size of the response and is rescaled to the data's own range each frame, which is what MATLAB's automatic `caxis` does — against a fixed ceiling the surface only ever samples the blue end and looks washed out.

Do not add per-frame motion. Every speed in the file is multiplied by `step`, the number of 60 Hz frames the last frame actually took. Without it a 120 Hz monitor runs the animation at double speed and 144 Hz at 2.4×. If you add anything that moves, multiply its increment by `step` as well.

The animation also pauses when the tab is hidden *and* when the canvas scrolls out of view, and it removes itself if the reader has asked for reduced motion.

8.2 The estimated IRF surface (irf-surface.js)

The 3-D surface is not decoration: it is the estimated response of inflation to an activity shock, from the hybrid TVP-BVAR with stochastic volatility in `Desktop/philips`. The numbers live in `irf-surface.js`, a generated file of about 5 KB that assigns one global, `window.__IRF_SURFACE__`, and is loaded by `index.qmd` *before* `background.js`.

It is delivered as a script rather than JSON on purpose: `fetch()` is blocked under `file://`, so a JSON file would silently fall back to the invented surface whenever `_site/index.html` is opened from disk. If the file is missing altogether the homepage still works — it renders a parametric stand-in instead.

Regenerating it. Only needed if the model is re-estimated.

1. In MATLAB, load the chosen results file, redo STEP 7 of `run_phillips_curve.m` and save the array:

```
load('India/Results_modspec_74.mat');
[theta, hret] = shuffleparameters(store_beta, store_alp, ...
                                store_h, included_vars, p);
htmp = hret; model = 1; % is_sv = [1 1] -> stochastic volatility
impulses = impulseresp(model, n, p, theta, [], htmp, 20, 1:T, 1, ...
                       0, 0, 0, 0, 0, OnlyStationary);
save('irf_surface.mat', 'impulses', '-v7.3');
```

This takes about 75 seconds. The MCMC itself is *not* re-run.

2. In R, read that file and write the script. `impulses` is indexed (response, shock, horizon, time); response 2 is inflation and shock 1 is activity, because the runner assembles `data = [act_col, inf_data]`.

Three traps, all of which cost time once already.

- `comp_resp = 0` in `user_config.m` means the runner never computes IRFs at all. That is why no impulse responses are stored in the `Results_modspec_*.mat` files — they hold only the MCMC draws.
- Passing `ci = 0` returns a 4-D array, not the 5-D one that `make_figures.m`'s `impulses(i,j,:,:,1)` implies. Ask for credible intervals and the quantile dimension reappears.
- MAT v7.3 is HDF5, and the usual advice is that HDF5 stores dimensions reversed. **Do not transpose.** `hdf5r` already undoes the reversal on read: its dims match MATLAB's `size()` exactly, and applying `aperm()` silently turns the 21×28 surface into a 2×2 slice.
- `OnlyStationary = 1` discards roughly 5,300 of the 20,000 draws at each time point, so the surface is a median over about 14,700 draws. This is expected, not an error.

Heights are divided by the largest absolute response rather than shifted, so zero stays at zero and the genuinely negative part of the response dips below the base plane instead of being flattened onto it. Because estimated data does not evolve, the surface sweeps itself in once along the time axis and then holds.

8.3 Site styling (`styles.css`)

The `.section` background alpha (currently 0.80) controls how much of the animation shows through content sections. Scholar badges are styled by `.sbadge`.

The palette has three tiers — pick by contrast

The teal comes in three values, and **which one you use depends on how large the text is**, not on taste:

Token	On white	Use for
<code>-accent</code>	2.5:1	Fills, rules, borders. Never text.
<code>-accent-dark</code>	3.9:1	Large text, borders
<code>-accent-text</code>	6.2:1	Small text and links
<code>-ink</code>	14.7:1	Body text
<code>-ink-soft</code>	7.6:1	Secondary text

WCAG AA wants **4.5:1** for normal text. `-accent-dark` does not reach it, which is why `-accent-text` was added in August 2026 — `.paper-meta` had been setting 0.84rem uppercase text at 3.9:1, the hardest case to read. If you introduce a new colour, check it before using it for text; there are contrast checkers online, or in R:

```
lum <- function(hex) { v <- col2rgb(hex)[,1]/255
  f <- ifelse(v <= 0.03928, v/12.92, ((v+0.055)/1.055)^2.4)
  sum(c(0.2126,0.7152,0.0722)*f) }
(lum("#ffffff")+0.05) / (lum("#376a5d")+0.05) # 6.21
```

Overriding the Quarto theme

The `cosmo` theme ships a blue link colour (`#2761e3`), a blue callout accent and pink inline code — three accents fighting the teal. `styles.css` pulls all three onto the palette under `main a`, `main .callout-note` and `main code:not(.sourceCode)`. Buttons and the navbar set their own colours with `!important` and are unaffected.

The callout icon is a blue SVG recoloured with `hue-rotate`. Blue sits near 220°; -60deg lands on the teal. Rotating the other way turns it pink.

Reading measure

```
.page-layout-article main.content { max-width: 72ch; line-height: 1.7; }
```

Prose past roughly 72 characters a line is tiring to read, and Bootstrap's 1.5 line-height is tight for long citation lists. **The `.page-layout-article` prefix is essential**: the homepage carries `page-layout-custom` and its full-bleed sections must not be constrained. Check which class a page has by searching its rendered HTML for `id="quarto-content"`.

Two gotchas worth remembering

Listing ids are renamed

Quarto turns `::: {#course-pages}` into a div with id `listing-course-pages`. Styling `#course-pages` silently matches nothing — which is exactly how the Teaching page card went unstyled for a while. Style `#listing-course-pages`.

`.paper-meta` is uppercase

Fine for a short label such as the tech line on a software card; unreadable for a whole sentence. Sentences use `.soft-note`.

The publication list

The Research page is not a numbered list — it is a two-column grid built by `pub_entry_html()` in `R/helpers.R`: year in the left gutter, then title, authors, and venue/DOI in quieter type. The year prints only on the *first* paper of each year so runs group visually. Styling lives under `.pub-list`, `.pub-entry`, `.pub-year`, `.pub-title`, `.pub-authors` and `.pub-venue`. Titles are styled on `.pub-title`, `.pub-title` a together so a paper with a DOI looks identical to one without.

8.4 Checking how a change looks

`quarto preview` gives a live local server. To capture pages as images without opening a browser — useful for comparing before and after, or for checking a page you are not currently looking at:

```
$chrome = "C:\\Program Files\\Google\\Chrome\\Application\\chrome.exe"
& $chrome --headless=new --disable-gpu --hide-scrollbars '
--window-size=1440,1400 --virtual-time-budget=8000 '
--screenshot="shot.png" '
"file:///C:/Users/Abc/Desktop/academic-website/_site/research.html"
```

`--virtual-time-budget` gives the page time to run its JavaScript, which the homepage's animated background and the plotly charts need. Note that over `file://` the browser blocks remote images, so the footer's visitor-count badge shows as broken in these screenshots — that is an artifact of local capture, not a fault on the live site.

8.5 Section formatting (R/helpers.R)

How each Excel-driven section is printed lives here: publication section headings in `PUB_SECTIONS`, course level headings in `TEACH_LEVELS`, Data Corner settings in `ECON_YEARS` and `MOSPI_BASE` (Section 6), chart colours in `CHART_COLORS`.

8.6 Courses taught

Courses are grouped by level (*Undergraduate*, *Postgraduate*, *Ph.D.*) and run together into one semicolon-separated sentence each, rather than set as a table — the old table spent a whole column repeating three values down twelve rows. To add a course, add a row to *teaching_courses* with `institution`, `course_title` and `level`; it is filed under the right heading automatically and sorted alphabetically. A `level` code that is not UG, PG or PhD still prints, under its own raw code, so a typo shows up on the page rather than vanishing.

The same grouping is produced twice, by two pieces of code that must be kept in step: `TEACH_LEVELS` plus `print_teaching()` in `R/helpers.R` for the website, and the Teaching chunk inside `cvnain/Dr Nain_CV.qmd` for the PDF. If you rename a heading, rename it in both.

9 Hosting on GitHub Pages

The GitHub account is <https://github.com/alignain>.

9.1 Choosing the repository name — this fixes the URL

Two options, and they give different addresses:

Repository	Site address	Notes
<code>alignain.github.io</code>	<code>https://alignain.github.io/</code>	The clean, root-level address. One per account. Recommended.
<code>website</code> (any name)	<code>https://alignain.github.io/website/</code>	Everything sits under a sub-path.

Pick before the first push: changing later means every link anyone has saved breaks. The rest of this section assumes `alignain.github.io`.

9.2 First-time setup

Step 0 — install git. It is not currently on this machine — rechecked August 2026: no `git`, no `gh`, and nothing bundled inside RStudio. Either

```
winget install --id Git.Git -e
```

or take the installer from <https://git-scm.com/download/win> and accept the defaults. Reopen the terminal afterwards so `git` is on the `PATH`. Then set your identity once:

```
git config --global user.name "Md Zulquar Nain"
git config --global user.email "alignain@gmail.com"
```

Step 1 — create the repository. On <https://github.com/new>, owner `alignain`, name `alignain.github.io`, visibility **Public** (Pages needs a paid plan to publish from a private repository). Add *no* `README`, *no* `.gitignore` and *no* licence — the folder already has what it needs, and an initial commit on GitHub’s side only causes a conflict on the first push.

Step 2 — push the folder. From `academic-website`:

```
git init
git add .
git status --short | measure          # sanity check: how many files?
git commit -m "Academic website"
git branch -M main
git remote add origin https://github.com/alignain/alignain.github.io.git
git push -u origin main
```

The first push carries roughly 50 MB and will take a few minutes. Git will ask you to sign in to GitHub in a browser window; that is normal, and it stores the credential so later pushes are silent.

Step 3 — turn on Pages. On GitHub: *Settings* → *Pages* → *Build and deployment* → *Source* → **GitHub Actions**. **Nothing is published until this is set**, and the deploy step of the workflow will fail with a permissions error if it is left on the default.

There is no `gh-pages` branch. The workflow renders the site, uploads `_site` as a Pages artifact and deploys from that, so the built output never enters the repository history. Do *not* run `quarto publish gh-pages`: it would create a branch that competes with the Actions deployment.

Step 4 — watch the first build. The *Actions* tab shows *Publish website to GitHub Pages* running. Expect **10–20 minutes** the first time: it installs R and twelve packages, then downloads about 115 MB of WebAssembly runtime to rebuild the Shinylive app. Later builds reuse a cache and take a few minutes.

Step 5 — the address is already recorded. This step used to say “set `site-url` once the run is green”. It is now set in `_quarto.yml` ahead of the first push:

```
site-url: https://alignain.github.io/
```

so there is nothing to do here and no second push. Quarto uses it for absolute links and social previews, and it is what makes Quarto emit `_site/sitemap.xml` and `_site/robots.txt` at all — with `site-url` unset, neither file is written and search engines get no index. If you ever change the repository name, change this line to match, or every address in the sitemap will point somewhere that does not exist.

9.3 Every update afterwards

```
git add .
git commit -m "Update publications"
git push
```

The included GitHub Action (`.github/workflows/publish.yml`) installs R, the required packages (`readxl`, `dplyr`, `glue`, `plotly`, `httr2`, `jsonlite`, `rvest`, `httr`, `tidyr`, `knitr`, `rmarkdown`, `shinylive`), and Quarto, rebuilds the Shinylive app, re-renders the site in the cloud, and deploys it. The first cloud build takes several minutes — it also downloads roughly 115 MB of WebAssembly runtime for the Shinylive app — while later builds reuse a cache and are faster.

9.4 What to commit

Commit everything except what `.gitignore` already excludes (`_site/`, `.quarto/`, R session files, `archive/`, `.claude/` — per-machine editor settings, not project content — and `software/splicer-app/` — 115 MB of generated WebAssembly that the cloud build regenerates from `software/app.R` with `software/build-shinylive.R`).

This guide is the one exception worth spelling out, because it is the only hand-run $\text{\LaTeX}$ project in the repository. Its `.tex` source *and* the built `Website_Maintenance_Guide.pdf` are both committed — the PDF is the copy anyone actually opens, and no cloud build compiles it — while `pdflatex`’s scratch files (`.aux`, `.log`, `.out`, `.toc`, and the `.fls/` `.fdb_latexmk/` `.synctex.gz` that `latexmk` and editor integrations add) are ignored. Those are rewritten on every compile and mean nothing to anyone else. The rule is written as `/guide/*.aux` and so on rather than a bare `*.log`, which would quietly swallow unrelated logs elsewhere in the project.

After editing the guide, recompile it *twice* — the second pass is what resolves the table of contents and the `Section n` cross-references — and commit the new PDF alongside the source:

```
cd guide
pdflatex -interaction=nonstopmode Website_Maintenance_Guide.tex
pdflatex -interaction=nonstopmode Website_Maintenance_Guide.tex
```

`pdflatex` is MiKTeX’s, installed under `%LOCALAPPDATA%\Programs\MiKTeX`.

`archive/` holds everything not needed to build or host the site: the legacy `cv/` project, the duplicate deck render, and R session junk. Nothing there was deleted, and `archive/README.md` says what each item is and how to put it back.

In particular *do* commit `irf-surface.js` (the estimated surface — it is generated, but from a 75-second MATLAB step that the cloud build cannot run), `data/*.csv` caches and `cvnain/` (the CV project and its Excel file are needed by the cloud build), and `Rclass_2025/` minus its own `_site/` sub-folder — the cloud build copies those slides straight into the published site, so if they are not in git they will not be online.

The first push carries roughly 53 MB and will take a few minutes (Section 3.6). Later pushes only carry what changed, so they are quick.

9.5 Visitor analytics

Counting is done by GoatCounter; the dashboard is <https://alignain.goatcounter.com>. It is privacy-preserving, needs no cookie banner, and the free tier is ample for a personal site.

Where the tag is defined. Once, in `_quarto.yml` under `format > html > include-in-header`. Nothing else should ever hard-code the endpoint — see the hook below, which reads it back out of that file precisely so the two cannot drift apart.

Why a post-render hook exists. `include-in-header` only reaches pages that *Quarto renders* — nine of them. The rest of the site is finished HTML that Quarto merely *copies*: the fourteen `Rclass_2025` lecture decks, the hand-rendered inflation tracker, the Shinylive splicer app. Those were being served untracked. `R/post-render.R` walks `_site` after every render and splices the tag into any page that lacks it:

<code>post-render: in _quarto.yml</code>	Pages counted
commented out	10 of 39
enabled	39 of 39

It is safe to re-run: pages that already carry the tag are skipped, so nothing is ever injected twice. It works on raw bytes rather than `readLines()/writeLines()` because the lecture decks are not all UTF-8 and a read-then-write round trip would re-encode them.

The footer badge. The counter in the page footer is written as a raw `<img>` in `_quarto.yml`, *not* as a markdown image:

```
right: ''
```

Written the markdown way, `![alt](url)`, Quarto wraps it in a `<figure>` and prints the alt text as a visible caption — so the footer showed the words “visitor count” underneath a badge that already read “Views for this site”. As raw HTML the text survives in `alt` for screen readers without being drawn twice.

The badge SVG hard-codes purple (`#9a15a4`) for its border and text and ignores a `?style=` parameter — the server returns the same colours regardless — so it is toned down from our side, in `styles.css` under `.visitor-badge`: `grayscale(1)`, `opacity 0.5` and a 38 px height, lifting to 0.85 on hover for anyone who actually wants to read the number.

Two things that look like faults and are not.

- **The badge reads 0 locally.** `count.js` deliberately ignores `localhost` and `file://`, so no local preview has ever registered a visit. The number only becomes real once the site is hosted.
- **The badge was a broken image.** The `/counter/` endpoint returns HTTP 403 until *Settings* → *allow public access to the counter* is ticked in GoatCounter. That has been enabled; if the badge ever reverts to a broken-image icon, check that setting first.

10 Troubleshooting

Stats badges show NA or old numbers

Google blocked the live fetch and no cache exists. Update the *scholar_fallback* sheet, or render again later (a successful fetch writes the cache and fixes it permanently).

PermissionDenied ..._site during render

Something (antivirus, sync client, an open file) is locking `_site`. Close it, delete `_site`, render again. This is chronic inside Google Drive folders — keep the project out of Drive.

Unable to locate an installed version of R

Set `QUARTO_R` as in Section 4, or render from RStudio.

A page renders empty, and the log says Sheet '...' is missing

A sheet was deleted or renamed in `cv_data.xlsx`. `R/helpers.R` deliberately warns and renders that section empty rather than killing the whole render, so **watch the render log** — the page will look fine apart from the missing content. Re-add the sheet with its original name and column headings. (This is how the *software* sheet went missing in July 2026, emptying the Software page — restored in August 2026. Every rewrite of the workbook leaves a dated backup beside it in `cvnain/data/`, which is what to fall back on.)

A CSS rule has no effect at all

First suspect the selector, not the CSS. Quarto renames listing containers (`#course-pages` becomes `#listing-course-pages`), and a rule aimed at the wrong id fails *silently* — nothing errors, the page just renders unstyled. Open the rendered file in `_site/` and search for the class or id you are targeting before assuming the rule is wrong.

A slide link works locally but 404s on the live site

Almost always capitalisation — see Section 3.3. Windows ignores it; the GitHub Pages server does not.

The Action is green but the site is 404

Settings → *Pages* → *Source* is still on *Deploy from a branch*. It must be **GitHub Actions**. Nothing is published until it is.

The deploy step fails with a permissions error

Same cause as above — `actions/deploy-pages` cannot create a deployment while Pages is set to branch mode.

git push rejected: *updates were rejected*

The GitHub repository was created with a README or licence, so it has a commit the local folder does not. Easiest fix is to delete the repository and recreate it empty; alternatively `git pull -rebase origin main` first.

The homepage surface looks smooth and invented rather than estimated

`irf-surface.js` did not load. Check it is committed, that `_quarto.yml` still lists it under `resources`, and that `index.qmd` loads it *before* `background.js`.

git push rejected: file exceeds 100 MB

Nothing in the project is near that any more (Section 3.6). If it happens, something generated has been committed by mistake — check that `software/splicer-app/` is still ignored.

The splicer app is live but GoatCounter shows no visits for it

`shinylive::export()` writes its own `index.html`, so the analytics tag has to be re-applied after every export. `software/build-shinylive.R` does this automatically, and `R/post-render.R` catches it again at the end of every render (Section 9.5); if the page was rebuilt some other way, re-run that script.

Lecture decks or the tracker are not being counted

`post-render:` is commented out in `_quarto.yml`. Coverage drops from 39 pages to 10 — the copied pages are the ones that lose the tag (Section 9.5). The render log prints the count every time: `post-render: analytics tag -- ... added`.

Cloud build fails on a missing R package

Add the package name to the `install.packages(...)` line in `.github/workflows/publish.yml`. It currently installs `readxl`, `dplyr`, `glue`, `plotly`, `httr2`, `jsonlite`, `rvest`, `httr`, `tidyr`, `knitr`, `rmarkdown`, `shinylive`.

Data Corner chart is stale or empty

The render log says `MoSPI: live fetch failed ... using cache`. The API was unreachable and `data/mospi_cache.csv` was drawn instead; if that file is missing too, the chart cannot render. Re-render when the API is back — one success repopulates the cache (Section 6).

CV renders but the site still serves an old cv.pdf

The site copies `cvnain/Dr-Nain-CV.pdf` at render time (`R/pre-render.R`); re-render the site after re-rendering the CV.